ASTANA IT UNIVERSITY • SOFTWARE ENGINEERING • SHP-2216

Visitor Pattern — Advanced File System

Assignment 06 • Technical & Architectural Defense Report

Format: Individual | **Language:** Java 17 (Maven)
Weight: 15% RK2 | **Defense:** Week 10

STUDENT	GROUP	TEST SUITE	REPOSITORY
Nursultan Maratov	SE-2523	8 / 8 Passing (100%)	github.com/nursultanmaratov/...

1. PURPOSE & ARCHITECTURAL DESIGN CHOICES

This project implements an extensible in-memory hierarchical file system using the **Visitor behavioral design pattern** in Java 17. The architecture separates the composite object structure (`Element`, `FileNode`, `DirectoryNode`) from operations executed over the tree (`TotalSizeVisitor`, `SearchVisitor`, `StatisticsVisitor`).

1.1 Double Dispatch Mechanism (CLO 1)

Standard Java resolves method invocations based only on the runtime receiver type (single dispatch). The Visitor pattern implements **Double Dispatch** across two coordinated steps:

1. **First Dispatch (Dynamic Polymorphism on Receiver):** The client calls `element.accept(visitor)`. Java resolves `accept()` dynamically at runtime on the concrete element type (`FileNode` or `DirectoryNode`).
2. **Second Dispatch (Static Overload Resolution + Dynamic Visitor Dispatch):** Inside `accept()`, the element calls `visitor.visit(this)`. The compiler statically binds `this` to the corresponding overloaded signature (`visit(FileNode)` or `visit(DirectoryNode)`), while the runtime selects the concrete visitor implementation.

This completely eliminates runtime type discrimination (`instanceof`, `getClass()` branching, reflection, or downcasting).

1.2 Traversal Ownership Justification (R7)

Choice: Visitors as Traversal Owners. Each visitor controls its own child traversal order and manages contextual state: `SearchVisitor` maintains a `Deque<String>` path stack to format deterministic root-relative paths (`docs/spec.pdf`); `StatisticsVisitor` tracks nesting depth levels (`currentDepth`, `maxDepth`); `TotalSizeVisitor` performs depth-first recursive accumulation. Element nodes remain lightweight composite containers focused solely on tree structure and integrity invariants.

1.3 Controlled State & Repeated Executions (T7)

To guarantee that repeat executions do not leak or multiply state, each visitor tracks its active `traversalDepth`. Upon root entry (`traversalDepth == 0`), previous results are automatically purged via an internal `reset()` call. Running a visitor twice on the same structure yields identical, idempotent results.

1.4 Tree Integrity Safeguards (R6)

`DirectoryNode` and `FileNode` predictably reject invalid mutations: negative file sizes and blank names (`IllegalArgumentException`), duplicate sibling names (`IllegalArgumentException`), multiple parent ownership (`IllegalStateException`), and cycle-forming links detected via parent pointer chain traversal (`IllegalArgumentException`). Encapsulation is preserved by exposing children as an unmodifiable list.

2. CLEAN CODE PRACTICES & DESIGN EXPLANATION

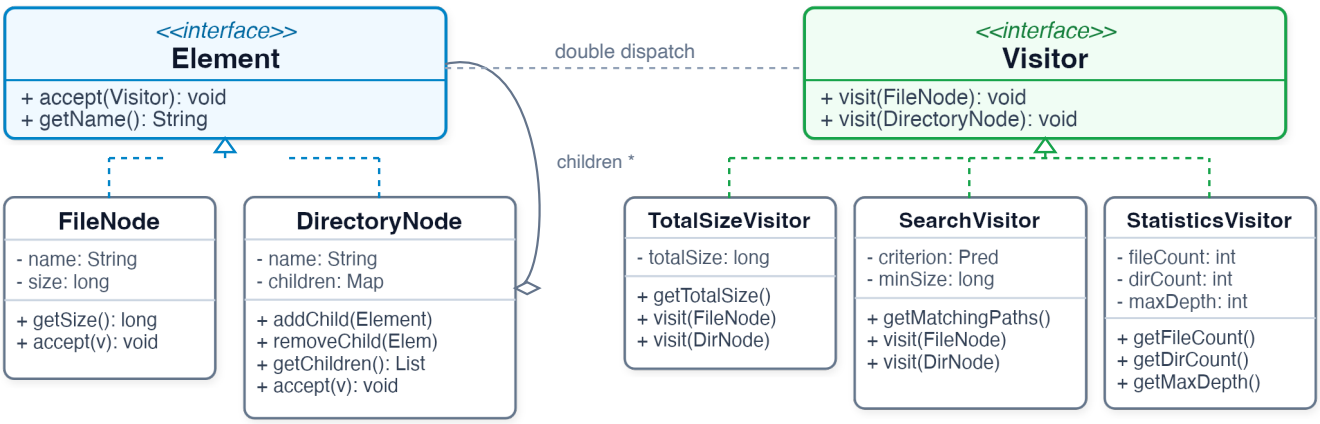
- **Meaningful Naming (`TotalSizeVisitor`, `SearchVisitor`, `StatisticsVisitor`):** Names clearly convey domain roles and visitor functions without cryptic abbreviations.
- **Single Responsibility Principle (Element vs. Visitor Hierarchy):** Elements exclusively manage tree topology and structural invariants; all computational algorithms reside inside discrete visitor classes.
- **Small Methods (`FileNode.accept()`, `DirectoryNode.accept()`):** Each method performs a single task; `accept()` is a single-line delegation (`visitor.visit(this);`).
- **Encapsulation (`DirectoryNode.getChildren()`):** Internal child collections are private; public callers receive an unmodifiable view via `Collections.unmodifiableList()`.
- **Elimination of Type-Checking Chains:** Zero `instanceof` or class-casting conditional chains exist; polymorphism handles element-to-operation binding cleanly.

Architectural Trade-off: Adding New Elements vs. New Operations

Adding new operations (Strength): Perfectly respects the Open/Closed Principle. Adding `StatisticsVisitor` (R8) required zero modifications to existing element classes.

Adding new element types (Trade-off): Introducing a new node (e.g. `SymlinkNode`) forces modifying the `Visitor` interface (`void visit(SymlinkNode)`), requiring updates across *all* existing concrete visitor implementations.

3. UML CLASS DIAGRAM



4. SAMPLE PROGRAM EXECUTION OUTPUT

Executed via `mvn exec:java` against a deterministic multi-level project tree:

```
================================================================================
   ASSIGNMENT 06: VISITOR PATTERN - ADVANCED FILE SYSTEM DEMONSTRATION
================================================================================
--- 1. FILE SYSTEM STRUCTURE VISUALIZATION ---
└── [DIR]  university-project/
    ├── [FILE] README.md (850 bytes)
    ├── [FILE] pom.xml (2400 bytes)
    └── [DIR]  docs/
        ├── [FILE] specification.pdf (45000 bytes)
        ├── [FILE] architecture.md (3200 bytes)
    └── [DIR]  src/
        ├── [FILE] Main.java (4800 bytes)
        ├── [FILE] Controller.java (1850 bytes)
        └── [DIR]  assets/
            ├── [FILE] logo.png (15400 bytes)

--- 2. OPERATION 1: TOTAL SIZE VISITOR (R3) ---
Total file size for entire tree ('university-project'): 73500 bytes | Subtree ('src'): 22050 bytes

--- 3. OPERATION 2: SEARCH VISITOR (R4, Deterministic & Root-Relative) ---
Search A: Files containing '.java' with size >= 2,000 bytes -> src/Main.java
Search B: All markdown files ('.md') with size >= 0 bytes   -> README.md, docs/architecture.md
Search C: Non-matching query ('non_existent')              -> (No matching files found)

--- 4. OPERATION 3: STATISTICS VISITOR (R5, Extensible OCP Operation) ---
Subtree statistics for root: 7 files, 4 directories (including root), maximum tree depth = 3

--- 5. REUSABILITY & REPEAT RUN VERIFICATION (T7) ---
Re-running TotalSizeVisitor: 73500 bytes (No leakage/doubling)
Re-running StatisticsVisitor: 7 files, 4 dirs, depth 3 (No multiplied counters)

--- 6. TREE INTEGRITY & INVARIANT VALIDATION (R6, T8) ---
  [PASS] Reject Negative File Size: Successfully caught IllegalArgumentException
  [PASS] Reject Duplicate Sibling Name: Successfully caught IllegalArgumentException
  [PASS] Reject Multiple Parent Ownership: Successfully caught IllegalStateException
  [PASS] Reject Cycle (Self-Addition): Successfully caught IllegalArgumentException
  [PASS] Reject Cycle (Ancestor to Descendant): Successfully caught IllegalArgumentException
================================================================================
```

5. AUTOMATED TEST SUITE SUMMARY (JUNIT 5)

The test suite in `kz.aitu.visitor.FileSystemVisitorTest` executes via `mvn test`, validating all eight mandatory checkpoints:

Check	Automated Target	Verification & Assertion Detail	Status
T1	Overloaded visit dispatch	Verifies `FileNode` and `DirectoryNode` dispatch to the correct overloaded visit methods.	**PASS** ✅
T2	Size accumulation	Calculates 25,500 bytes across 3 nesting levels; validates subtree (13,000 bytes) and leaf.	**PASS** ✅
T3	Combined search criteria	Combines filename filter (`.java`) and threshold (`>= 2000`); correctly includes/excludes.	**PASS** ✅
T4	Deterministic root-relative paths	Validates `/` separator, root-relative path format, and lexicographical sorting.	**PASS** ✅
T5	Statistics metrics	Confirms 5 files, 4 directories (including root), and maximum depth of 3 (root depth = 0).	**PASS** ✅
T6	Empty structures & edge cases	Validates empty dir (size 0, 0 files, 1 dir, depth 0) and non-matching query (empty list).	**PASS** ✅
T7	Controlled state reusability	Repeated runs on the same visitor instance do not leak, double, or corrupt accumulated state.	**PASS** ✅
T8	Tree integrity & invariants	Rejects negative sizes, blank names, duplicate siblings, multi-parenting, and cycles.	**PASS** ✅

Test Runner Output: `Tests run: 8, Failures: 0, Errors: 0, Skipped: 0. BUILD SUCCESS (1.079 s)`

6. INDIVIDUAL DEFENSE PREPARATION & VERIFICATION CHECKLIST

- **Step 1 — Trace Two Dispatch Steps:**
  1. `element.accept(v)` : dynamic dispatch in `FileNode.java` (line 52) or `DirectoryNode.java` (line 152).
  2. `v.visit(this)` : static compile-time overload binding to `visit(FileNode)` or `visit(DirectoryNode)` + dynamic runtime dispatch on concrete visitor.
- **Step 2 — Run Full Test Suite:** Execute `mvn test` from the project root; all 8 test cases pass synchronously without IDE dependency.
- **Step 3 — Input/Search Configuration Change:** Demonstrate how changing search predicate or minimum size threshold instantly filters outputs in `Main.java` or `SearchVisitor`.
- **Step 4 — Architectural Trade-off Defense:** Explain that adding operations adheres to OCP (proven by Commit `cc6eaf6` adding `StatisticsVisitor` with zero element changes), while adding new element types violates OCP by forcing updates across all visitor implementations.

7. GITHUB REPOSITORY LINK

The complete source code, commit history showing progressive visitor addition, and tests are available at:
GitHub Repository: https://github.com/nursultanmaratov/Assignment6_Visitor_SE2523_Maratov_Nursultan